YUVAINTERN – MACHINE LEARNING INTERNSHIP

Weeks 2–6: Technical Strategy and Documentation Report

Project: AI-Powered Urban Heat Reduction System Using IoT

Prepared for: YuvaIntern
Technology: Python, IoT sensor data, Machine Learning

This document presents a connected plan for data exploration, preprocessing, model architecture, training and tuning, performance evaluation, and deployment.

Project Overview

The project aims to support urban heat monitoring and reduction by combining environmental measurements from IoT sensors with machine-learning predictions. A network of sensors can collect variables such as air temperature, relative humidity, surface temperature, light intensity, and—where available—wind speed and air quality indicators. The model can use these observations, together with time and location features, to estimate near-term heat conditions and help identify areas that may require mitigation.

The intended prototype is a decision-support system, not an automated authority for public-safety decisions. Its predictions should be displayed with timestamps, sensor-quality indicators, and uncertainty or confidence information where possible. Potential actions such as increasing shade, adding vegetation, or adjusting cooling interventions should be reviewed by relevant people and evaluated using local evidence.

Important planning assumption: No actual dataset, sensor inventory, city boundary, or deployment hardware specification was provided. Therefore, the report defines a practical strategy and candidate features. Final choices must be confirmed after inspecting the real dataset and project requirements.

Proposed Problem Definition

Primary ML task: supervised regression to predict air temperature (°C) at a sensor location for a defined future interval, such as the next hour. A secondary classification task may label conditions as normal, elevated, or high heat using thresholds approved for the project location. The regression task is the main model-design example in this report.

Candidate Data Fields

Field	Meaning / use
timestamp	Date and time of sensor reading; derive hour, day, month, and season.
sensor_id / location_id	Sensor identity or site; useful for grouping and spatial validation.
latitude / longitude or zone	Location context; protect sensitive site information where applicable.
air_temperature	Main target for prediction, or a lagged input for future prediction.
relative_humidity	Environmental predictor.
surface_temperature	May indicate heat stored by roads, roofs, or other surfaces.
light_intensity / solar exposure	Proxy for incoming sunlight, depending on sensor capability.
wind_speed	Optional predictor if measured reliably.
vegetation / land-cover indicator	Optional contextual feature from a trusted source.
sensor_status / battery	Quality-control information; usually not a predictive feature.

End-to-End Workflow

IoT sensors and contextual data → data validation and storage → exploratory data analysis → preprocessing and feature creation → train/validation/test split → model training and tuning → independent evaluation → prototype deployment → monitoring and periodic maintenance.

Week 2 – Exploratory Data Analysis and Preprocessing Strategy

1. Objective and Scope

The purpose of Week 2 is to understand the structure, quality, and usefulness of the available data before model development. The analysis will establish whether the sensor readings are sufficiently complete, consistent, representative, and reliable for predicting urban temperature. All decisions will be recorded so that the preprocessing process can be repeated.

2. Step-by-Step EDA Plan

• **Load and inspect:** read CSV or database exports with pandas; inspect file size, column names, data types, sample rows, and summary statistics using head(), info(), describe(), and shape.

• **Check data integrity:** identify duplicate records, missing timestamps, invalid sensor IDs, impossible values, inconsistent units, and repeated or irregular sampling intervals.

• **Validate ranges:** compare temperature, humidity, wind, and other measurements against sensor specifications and plausible local environmental ranges. Flag suspicious values for review rather than deleting them automatically.

• **Study distributions:** use histograms and density plots for numeric variables; use count plots for categorical fields; compare distributions by sensor, location, time of day, and season.

• **Inspect outliers:** use box plots, time-series plots, and robust statistics such as median and interquartile range. Distinguish genuine heat events from sensor faults.

• **Explore relationships:** use scatter plots and correlation heatmaps to examine relationships among temperature, humidity, solar exposure, surface temperature, and wind speed.

• **Check temporal and spatial coverage:** plot readings over time, compare sensors, identify gaps, and check whether some areas or time periods are under-represented.

• **Document findings:** create a data dictionary, data-quality summary, and a record of all preprocessing decisions.

3. Data Cleaning and Missing Values

Exact duplicate rows should be removed after checking whether repeated timestamps represent genuine repeated measurements. Column names and units should be standardized. Timestamps should be parsed into a consistent time zone and sorted. Invalid records should be flagged using sensor specifications and domain checks. Missingness should be measured by column, sensor, location, and time period.

• **Small isolated gaps:** consider time-based interpolation only when readings are regularly sampled and the gap is short. Keep an indicator showing that the value was imputed.

• **Moderate missingness:** use median imputation for skewed numeric variables or a training-set-fitted imputer. Use the most frequent value or an explicit 'Unknown' category for categorical fields.

• **Long gaps or faulty sensors:** avoid inventing values across long periods; exclude affected windows or sensors when justified and document the loss of coverage.

• **Target values:** do not impute missing target temperature values for supervised training. Exclude those target rows from model fitting and report the amount removed.

• **Leakage prevention:** fit imputers, scalers, and encoders on training data only, then apply the fitted transformations to validation and test data.

4. Outlier Treatment

Outliers will first be investigated with plots and sensor metadata. A sudden temperature spike may be a genuine heat event and should not be removed solely because it is statistically unusual. Confirmed sensor errors can be corrected from source records or marked missing. If extreme but valid observations are retained, robust models or robust scaling can reduce their influence. Winsorization or clipping should be used only with a documented physical or operational reason.

5. Feature Engineering and Transformation

• Create hour, day of week, month, and season from timestamps; encode cyclical time using sine and cosine transformations when useful.

• Create lag features (for example, previous 1-hour and 3-hour temperature) and rolling means only from past observations.

• Create sensor/location features and optional land-cover or vegetation indicators when trustworthy data are available.

• Encode categorical variables using one-hot encoding or another suitable encoding; scale numeric features for algorithms that are sensitive to feature scale.

• Avoid using future readings, post-event information, or identifiers that merely memorize the target.

6. Visualizations and Profiling Outputs

Visualization / method	Purpose
Histograms	Understand distributions, skewness, and unusual values.
Box plots	Compare spread and possible outliers across variables or sensors.
Time-series line plots	Reveal trends, cycles, sensor dropouts, and abrupt changes.
Scatter plots	Inspect relationships between temperature and environmental variables.
Correlation heatmap	Identify linear associations and potential multicollinearity.
Missing-value matrix / bar chart	Show the amount and pattern of missing data.
Sensor-wise comparison plots	Find calibration differences and spatial coverage gaps.

7. Challenges and Proposed Solutions

Challenge	Proposed response
Sensor drift or calibration differences	Check calibration records; compare overlapping sensors; flag suspicious shifts.
Missing or irregular readings	Quantify gap patterns; use limited time interpolation or training-fitted imputation; exclude long faulty periods.
Real heat extremes mistaken for errors	Review time series and contextual readings before filtering.
Seasonal or location imbalance	Use stratified/group-aware analysis and report coverage limitations.
Data leakage from time features	Use chronological splits and create lag/rolling features using past data only.
Different measurement units	Standardize units and record conversions in the data dictionary.

8. Week 2 Time Plan (6 Hours)

Activity	Hours
Load data, understand schema, and create data dictionary	1.0
Quality checks: missingness, duplicates, ranges, timestamps	1.5
Univariate and multivariate visualizations	1.5
Preprocessing and feature-engineering plan	1.0
Document findings and risks	1.0
Total	6.0

9. Submission Description (200+ Words)

This report presents a structured exploratory data analysis and preprocessing strategy for the AI-Powered Urban Heat Reduction System Using IoT. The objective of this stage is to understand whether the collected environmental and sensor data are suitable for developing a reliable machine-learning model. The proposed process begins with loading the dataset in Python and examining its dimensions, column names, data types, sample records, and statistical summaries using pandas. It then checks data quality issues such as missing values, duplicate readings, invalid timestamps, inconsistent units, unusual sensor measurements, and uneven sampling intervals. The report explains how histograms, box plots, time-series charts, scatter plots, missing-value visualizations, and correlation heatmaps can be used to understand distributions and relationships between environmental variables. It also describes practical methods for treating missing values, investigating outliers, standardizing units, and identifying faulty sensor readings without accidentally deleting genuine high-temperature events. Feature engineering ideas include extracting time-based variables, creating lagged readings, calculating rolling statistics, and adding location or land-cover information when reliable sources are available. The strategy emphasizes preventing data leakage by fitting preprocessing steps only on training data and by using chronological or group-aware data splits where appropriate. Potential challenges, including sensor drift, missing data, location imbalance, and seasonal changes, are paired with realistic solutions. A six-hour activity plan is included to keep the work aligned with the internship schedule. Since the final dataset has not yet been specified, the methods are presented as a practical plan that will be confirmed and adjusted after inspecting the actual sensor data. The outcome of this stage will be a documented, repeatable dataset preparation process that provides a sound foundation for model design and training.

Week 3 – Model Architecture Design and Development

1. Objective

Week 3 converts the project problem and Week 2 data findings into a model blueprint. The main prototype will predict future air temperature at a sensor location. This is a regression problem because the output is a continuous numeric value measured in degrees Celsius.

2. Selected Model Approach

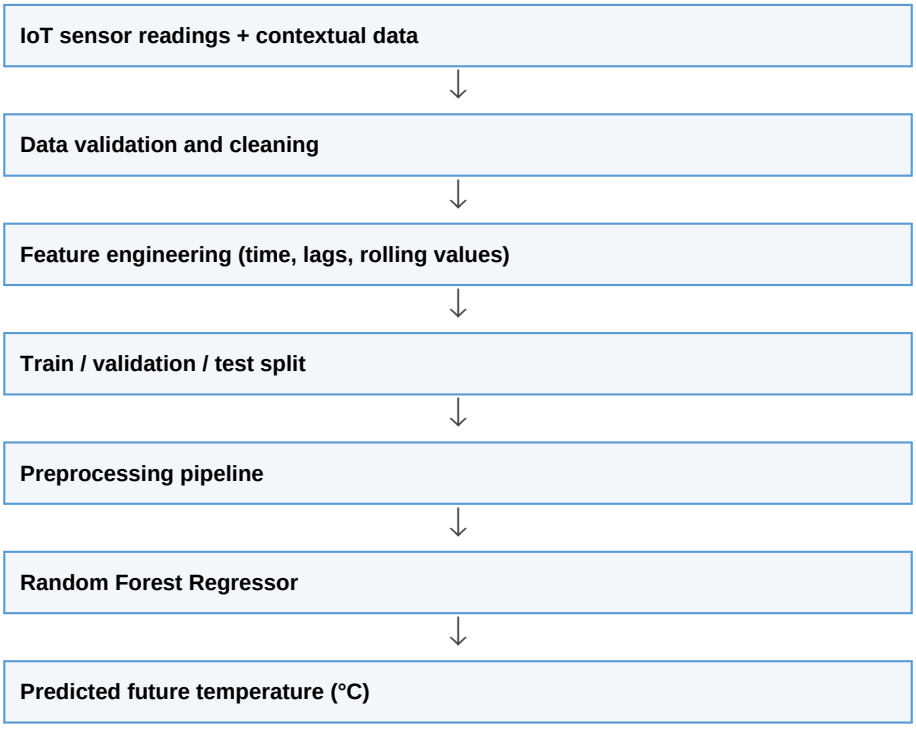
Primary candidate: Random Forest Regressor. Random Forest is a practical baseline for tabular environmental data. It can model nonlinear relationships and interactions, requires limited feature scaling, and provides feature-importance estimates that can help explain which inputs influence predictions. It is also relatively straightforward to train and evaluate within a short internship project.

Comparison candidate: Gradient Boosting Regressor (such as HistGradientBoostingRegressor or another available gradient-boosting implementation). It may improve predictive accuracy on structured data but requires careful tuning. A simple baseline such as predicting the latest observed temperature or the training-set mean should also be included to determine whether machine learning adds value.

3. Proposed Architecture

Stage	Component / action
Input	Validated IoT readings, time features, location/context features, and past-only lag features.
Preprocessing	Imputation, categorical encoding, optional scaling, and feature selection inside a reproducible pipeline.
Split	Chronological train/validation/test split; group by sensor or location where required.
Model	Random Forest Regressor as primary model; Gradient Boosting as comparison.
Output	Predicted air temperature for the selected future horizon, in °C.
Post-processing	Attach timestamp, sensor/location, input-quality flags, and optional prediction interval or uncertainty estimate.
Evaluation	MAE, RMSE, R^2, residual analysis, and comparison with a simple baseline.

4. Model Flow Diagram



↓

Evaluation, visualization, and decision-support display

5. Feature Integration and Assumptions

• The target is the air temperature at a defined future horizon; the horizon must be fixed before training.

• Input measurements are timestamped and sufficiently frequent to support lag and rolling features.

• Sensor readings are calibrated or have quality metadata that can be used to identify unreliable observations.

• Training examples are representative of the locations and seasons where predictions will be used.

• The model will not use future observations or variables derived from the target after the prediction time.

• The initial prototype is a decision-support tool; it does not automatically control city infrastructure.

6. Limitations and Backup Strategies

Random Forest may be less effective at extrapolating beyond the temperature range represented in training data, and it may not capture all temporal dynamics. If performance is weak, compare gradient boosting, regularized linear regression, or a time-series approach. If data volume and quality later justify it, sequence models such as LSTM may be explored, but they are not the first choice for a limited prototype. If sensor locations have distinct patterns, use location-aware validation and consider separate local models only when sufficient data exist.

7. Week 3 Time Plan (6 Hours)

Activity	Hours
Review problem, target, and Week 2 findings	1.0
Compare candidate algorithms and select baseline	1.5
Design features, pipeline, and data flow	1.5
Prepare architecture diagram and assumptions	1.0
Document limitations and backup strategies	1.0
Total	6.0

8. Submission Description (200+ Words)

This report documents the proposed model architecture for the AI-Powered Urban Heat Reduction System Using IoT. It connects the project objective with the data preparation strategy and defines a clear blueprint for the model-training stage. The main machine-learning task is framed as supervised regression, where historical IoT sensor readings and contextual features are used to predict air temperature at a specified future time. A Random Forest Regressor is selected as the primary prototype because it can learn nonlinear relationships in structured environmental data, is relatively robust to different feature scales, and provides useful feature-importance information. A Gradient Boosting model is proposed as a comparison so that the final choice can be based on measured validation performance rather than assumption. The report describes the complete processing flow, beginning with validated sensor readings and continuing through feature engineering, data splitting, preprocessing, model fitting, prediction, and evaluation. It includes candidate input features such as temperature history, humidity, surface temperature, solar exposure, wind speed, time variables, and location information, subject to availability and data quality. A block-style architecture diagram illustrates the sequence of components and the expected model output. The assumptions section explains the need for reliable timestamps, representative sensor coverage, a fixed prediction horizon, and strict prevention of future-data leakage. The report also identifies limitations, including difficulty predicting conditions outside the training range and possible differences between sensor locations. Backup approaches such as gradient boosting, regularized regression, or later time-series models are discussed. This architecture is intended to serve as an executable plan for the next phase while remaining flexible enough to change after the actual dataset is inspected.

Week 4 – Model Training and Hyperparameter Optimization

1. Training Environment

The prototype will be developed in Python using Jupyter Notebook or Google Colab. Core libraries may include pandas and NumPy for data handling, Matplotlib and Seaborn for visualization, and scikit-learn for preprocessing, model training, validation, and metrics. Package versions and random seeds should be recorded so that results can be reproduced.

2. Data Splitting and Validation

Because sensor readings are time-dependent, a random split may place future observations in the training set and earlier observations in the test set. The preferred approach is a chronological split, for example 70% earliest observations for training, the next 15% for validation, and the latest 15% for final testing. Exact percentages can be adjusted to the amount of data. If the intended use is prediction at unseen locations, a group-based split by sensor or location should also be considered. The final test set must remain untouched until model selection and tuning are complete.

3. Training Pipeline

• Define the target and prediction horizon and remove rows without valid target values.

• Create features using only information available at the prediction timestamp.

• Build a scikit-learn Pipeline or ColumnTransformer for imputation, encoding, and optional scaling.

• Fit all preprocessing steps on training data only.

• Train a baseline model and the Random Forest model using the same split.

• Evaluate validation results, inspect residuals, and record metrics and parameters.

• Select the final configuration, refit on training plus validation data if appropriate, and evaluate once on the untouched test set.

4. Hyperparameters to Tune

Model	Candidate hyperparameters
Random Forest	n_estimators (100, 200, 400); max_depth (None, 10, 20); min_samples_split (2, 5, 10); min_samples_leaf (1, 2, 4); max_fe
Gradient Boosting	learning_rate (0.03, 0.05, 0.1); max_iter / n_estimators; max_leaf_nodes or max_depth; min_samples_leaf; l2_regularizati
Baseline	No meaningful tuning for a simple persistence or mean predictor; use it as a reference point.

5. Optimization Method

Start with a small RandomizedSearchCV or a manually defined randomized parameter sample to explore a broad range efficiently. Use time-aware validation rather than ordinary shuffled cross-validation. If the dataset supports it, use expanding-window or rolling-origin validation. After identifying promising ranges, run a smaller GridSearchCV around the best settings. Bayesian optimization can be considered only if the project environment supports it and the search space is large enough to justify the added setup.

6. Metrics and Overfitting Controls

• **MAE:** average absolute prediction error in °C; easy to interpret.

• **RMSE:** penalizes larger errors more strongly and highlights occasional large misses.

• **R^2:** indicates explained variance, but should not be used alone.

• Compare training and validation errors; a large gap may indicate overfitting.

• Limit tree depth, increase minimum leaf size, reduce unnecessary features, or collect more representative data if overfitting occurs.

• Record runtime and memory use; reduce search iterations or use fewer trees if computation is too slow.

7. Expected Outcome

The expected outcome is a reproducible training pipeline, a baseline score, a tuned candidate model, and a transparent record of the effect of each tuning choice. Improvement will be measured against the baseline on validation data and confirmed on the final test set. No accuracy or error improvement is guaranteed in advance; actual results must be calculated from the dataset.

8. Week 4 Time Plan (7 Hours)

Activity	Hours
Set up Python environment and reproducibility	1.0
Create chronological/group-aware splits and pipeline	1.5
Train baseline and initial models	1.5
Run randomized tuning and focused search	2.0
Review results and document risks	1.0
Total	7.0

9. Submission Description (200+ Words)

This report explains a systematic plan for training and optimizing the machine-learning model proposed for the AI-Powered Urban Heat Reduction System Using IoT. It describes the Python environment, the libraries required, the data preparation pipeline, and the validation strategy that will be used to obtain trustworthy model results. Since environmental sensor readings are time-dependent, the report recommends chronological data splitting so that the model is trained on earlier observations and validated on later observations. A group-aware split by sensor or location is also discussed for situations where the project must generalize to locations that were not included in training. The report explains why the final test set should remain separate until model selection is complete. A reproducible scikit-learn pipeline will combine missing-value handling, categorical encoding, optional scaling, and the selected regression algorithm. The Random Forest Regressor will be trained as the primary candidate, while Gradient Boosting may be used for comparison. Important hyperparameters such as the number of trees, tree depth, minimum samples per split, minimum samples per leaf, learning rate, and leaf complexity are identified. Randomized search is proposed for efficient exploration, followed by a smaller focused grid search when useful. Time-aware validation will be preferred over shuffled cross-validation to reduce leakage. MAE, RMSE, and R-squared will be used to measure performance, with training-versus-validation comparisons to identify overfitting. The report also provides contingency measures for long training times, excessive model complexity, and weak generalization. A seven-hour schedule keeps the work realistic within the internship plan. The final deliverable will be a reproducible model-training process and a documented selection of hyperparameters supported by validation evidence.

Week 5 – Performance Evaluation and Interpretation

1. Evaluation Objective

Week 5 determines whether the trained model predicts future temperature accurately and consistently enough to support the intended prototype. Evaluation will consider prediction error, generalization across time and sensors, robustness to imperfect inputs, and possible bias caused by uneven data coverage.

2. Evaluation Metrics

Metric	Meaning and use
MAE	Mean absolute error in °C; gives an intuitive average error.
RMSE	Root mean squared error in °C; gives greater weight to large errors.
R^2	Compares the model with a mean-target reference; can be negative on poor test predictions.
Bias / mean error	Shows whether the model systematically overpredicts or underpredicts temperature.
Segmented MAE	Measures error by sensor, location, hour, month, or temperature range to reveal uneven performance.

Accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices are classification measures. They should only be used if the project also defines a heat-category classification task. For that optional task, define locally appropriate and approved temperature thresholds, then report precision, recall, F1-score, confusion matrix, and ROC-AUC where the class structure permits.

3. Evaluation Procedure

• Freeze the selected pipeline and model configuration before opening the test set.

• Generate predictions for the untouched chronological test period.

• Calculate MAE, RMSE, R^2, and mean error; compare with the persistence or mean baseline.

• Plot actual versus predicted temperatures and residuals over time.

• Inspect errors by sensor, location, hour, month, and low/medium/high temperature range.

• Check whether errors increase during heat events, at night, during sensor outages, or at locations with limited training data.

• Record the dataset period, split boundaries, feature list, model version, and software versions.

4. Visual Interpretation

• **Actual-versus-predicted scatter plot:** points close to the diagonal indicate closer predictions.

• **Residual plot:** reveals systematic patterns, nonlinearity, and changing error spread.

• **Residual histogram:** shows the distribution and skew of prediction errors.

• **Time-series overlay:** compares actual and predicted temperature through heat peaks and daily cycles.

• **Segmented error chart:** compares model quality across sensors, locations, and time periods.

• **Confusion matrix / ROC curve:** use only for an explicitly defined heat-risk classification extension.

5. Error Analysis, Bias, and Corrective Actions

Observed issue	Possible action
Errors rise at one sensor	Check calibration, placement, missingness, and sensor-specific bias.
Model underpredicts heat peaks	Inspect peak-event coverage, lag features, and model capacity; retain valid extremes.
Errors vary by season	Add representative seasonal data and evaluate temporal features.

Observed issue	Possible action
Poor performance at unseen locations	Collect local data, use group-aware validation, or limit claims to supported locations.
Residuals show a time pattern	Review prediction horizon, lag features, and temporal validation.
Performance differs across groups	Report subgroup metrics and investigate data coverage or measurement quality.

6. Limitations

A single test period may not represent future weather conditions or unusual heat events. Sensor networks may be geographically uneven, and external factors such as construction, traffic, cloud cover, and changes in land use may not be captured. Metrics summarize historical performance and cannot guarantee future accuracy. Evaluation should therefore be repeated as new data arrive, with clear statements about where and when the model has been tested.

7. Week 5 Time Plan (6 Hours)

Activity	Hours
Finalize test protocol and metrics	1.0
Run test predictions and calculate metrics	1.0
Create evaluation visualizations	1.5
Conduct error and subgroup analysis	1.5
Document limitations and improvements	1.0
Total	6.0

8. Submission Description (200+ Words)

This report develops a complete performance-evaluation strategy for the machine-learning model planned for the AI-Powered Urban Heat Reduction System Using IoT. The evaluation is designed to determine not only whether the model produces low prediction errors, but also whether it generalizes across time periods, sensor locations, and different environmental conditions. Since the main task is regression, Mean Absolute Error, Root Mean Squared Error, R-squared, and mean prediction error are selected as the primary metrics. MAE provides an understandable average error in degrees Celsius, while RMSE gives additional weight to large mistakes that may be important during extreme heat conditions. R-squared provides a complementary view of explained variation, but it will not be interpreted alone. The report explains how to keep the final test set separate from model tuning and how to compare the selected model with a simple baseline. It proposes visual tools such as actual-versus-predicted plots, residual plots, residual histograms, time-series overlays, and subgroup error charts. These visualizations can reveal systematic underprediction, changing error spread, seasonal weaknesses, or sensor-specific problems. The error-analysis section considers possible bias caused by uneven geographic coverage, faulty sensors, missing readings, and under-representation of extreme heat events. It recommends investigating the source of each performance gap before making changes to the model or data. Classification metrics such as precision, recall, F1-score, ROC-AUC, and confusion matrices are included only for an optional heat-category classification extension, because they are not appropriate as primary measures for a continuous temperature prediction task. The report also acknowledges that historical test results cannot guarantee future performance and recommends repeated monitoring as new data become available. A six-hour work plan provides a realistic structure for completing the evaluation and documenting improvement opportunities.

Week 6 – Model Deployment Strategy and Final Documentation

1. Deployment Goal

The final stage proposes a simulated deployment in which validated IoT readings are processed by a saved machine-learning pipeline and predictions are made available through a simple dashboard or API. The initial deployment is a prototype for demonstration and evaluation, not a city-wide production service.

2. Proposed Deployment Architecture

Layer	Proposed component
Data acquisition	IoT sensors or a sample data feed provide timestamped environmental readings.
Ingestion and validation	Python script or lightweight API validates schema, units, timestamps, and sensor status.
Preprocessing	The same saved preprocessing pipeline used during training transforms incoming records.
Inference	Load the versioned model and generate the temperature prediction.
Storage	Store input metadata, predictions, timestamps, model version, and quality flags.
Presentation	A simple Streamlit dashboard or API displays current readings, predicted temperature, and trends.
Monitoring	Track missing inputs, prediction latency, sensor availability, prediction distribution, and later-observed error.

3. Simulated Deployment Steps

• Finalize the selected model and preprocessing pipeline; save them together using a suitable serialization method such as joblib.

• Create a clean project structure containing data preparation, training, inference, dashboard/API, configuration, and documentation files.

• Write an inference script that validates input columns and data types before applying the saved pipeline.

• Test the pipeline with sample IoT records, including missing fields, out-of-range values, delayed records, and unavailable sensors.

• Build a small Streamlit interface or REST API that displays the latest input readings and predicted temperature.

• Record model version, prediction timestamp, input sensor, and quality flags for traceability.

• Optionally package the application with Docker after the local prototype is stable.

4. Monitoring and Maintenance

• **Operational monitoring:** track sensor connectivity, missing readings, invalid values, processing failures, response time, and application availability.

• **Data monitoring:** compare incoming feature distributions with training data to identify drift or changes in sensor behavior.

• **Performance monitoring:** when actual future temperatures become available, calculate rolling MAE and RMSE and compare them with agreed thresholds.

• **Maintenance:** investigate sensor calibration, update dependencies, back up data, document model changes, and retrain only after reviewing new data and validation results.

• **Rollback:** keep the previous validated model version so that a problematic update can be reversed.

5. Deployment Risks and Mitigation

Risk	Mitigation
Sensor or network outage	Show stale-data warnings; retry ingestion; avoid presenting old predictions as current.

Risk	Mitigation
Unexpected input schema	Validate required fields and units; reject or quarantine invalid records.
Training-serving mismatch	Reuse the same serialized preprocessing pipeline for inference.
Data drift / seasonal change	Monitor distributions and rolling error; schedule review and retraining when justified.
Security and privacy concerns	Use access controls, secure communication, minimal location detail, and safe secret handling.
Application or model failure	Log errors, provide a safe fallback status, and retain a known-good model version.
Limited compute resources	Start with a lightweight local deployment; measure resource use before scaling.

6. Scalability and CI/CD Considerations

For a student prototype, local execution or a small cloud instance is sufficient. If the system expands, ingestion, inference, storage, and visualization can be separated into services. Docker can help make the environment repeatable. A basic CI workflow can run unit tests, schema checks, and a small inference test whenever code changes. Model promotion should require evaluation on a fixed validation process and human approval; a new model should not replace the current version automatically.

7. Final Project Summary

The project lifecycle begins by defining a temperature-prediction objective for an IoT-based urban heat monitoring system. Week 2 establishes a data-quality and preprocessing strategy. Week 3 selects a practical regression architecture and documents its assumptions. Week 4 defines reproducible training, time-aware validation, and hyperparameter tuning. Week 5 establishes suitable regression metrics, visual analysis, and error diagnosis. Week 6 connects these stages in a simulated deployment with input validation, model versioning, monitoring, and maintenance. Together, the stages create a traceable plan from raw sensor observations to a monitored prediction prototype.

8. Week 6 Time Plan (6 Hours)

Activity	Hours
Design deployment architecture and project structure	1.0
Prepare saved model and inference pipeline plan	1.5
Plan dashboard/API and simulated input tests	1.0
Define monitoring, maintenance, and rollback	1.0
Compile final report and project summary	1.5
Total	6.0

9. Submission Description (200+ Words)

This report presents a deployment strategy and final project-lifecycle summary for the AI-Powered Urban Heat Reduction System Using IoT. Its purpose is to explain how the machine-learning model can move from a development notebook into a controlled simulated environment where new sensor readings are validated, processed, and used to generate temperature predictions. The proposed architecture includes data acquisition, input validation, a reusable preprocessing pipeline, model inference, prediction storage, a simple dashboard or API, and monitoring components. The deployment plan recommends saving the fitted preprocessing and model together so that incoming records receive the same transformations used during training. A lightweight Streamlit dashboard or REST API can display recent sensor measurements, predicted temperature, timestamps, and data-quality warnings. The report outlines practical implementation steps, including organizing the Python project, testing sample records, handling missing or invalid fields, recording model versions, and optionally using Docker to improve environment consistency. It also discusses operational monitoring for sensor connectivity, missing data, invalid readings, application response

time, and prediction latency. When actual future temperature observations become available, rolling evaluation metrics such as MAE and RMSE can be used to identify declining performance. Risks including sensor outages, schema changes, data drift, security issues, training-serving mismatch, and model failure are paired with mitigation measures such as validation, access controls, logging, versioning, and rollback. Scalability and basic CI/CD practices are considered at a level appropriate for a student prototype. The report concludes by connecting the EDA, architecture, training, evaluation, and deployment stages into one coherent workflow. Since no real sensor dataset or production infrastructure was supplied, the document describes a realistic simulated deployment plan rather than claiming that a live system has already been deployed.

Overall Internship Work Plan (31 Hours)

The following schedule totals 31 hours, which fits the stated 30–35-hour project window. The hours are planning estimates and can be adjusted after the dataset and available tools are confirmed.

Week	Focus	Hours
Week 2	EDA and preprocessing strategy	6
Week 3	Model architecture design	6
Week 4	Training and hyperparameter optimization	7
Week 5	Performance evaluation and interpretation	6
Week 6	Deployment strategy and final documentation	6
Total		31

Suggested Python Project Structure

```
urban-heat-iot-ml/
■■■ data/ # raw and processed data (do not commit private data)
■■■ notebooks/ # EDA and experiments
■■■ src/
■ ■■■ preprocessing.py
■ ■■■ features.py
■ ■■■ train.py
■ ■■■ evaluate.py
■ ■■■ predict.py
■■■ models/ # saved model artifacts (if permitted)
■■■ app/ # dashboard or API
■■■ tests/ # data and inference tests
■■■ requirements.txt
■■■ README.md
```

Important Submission Notes

• The task portal text specifies a Word document (.doc or .docx) for each weekly submission. This file is a PDF version for reading and reference; check whether the portal accepts PDF before uploading. If it enforces Word-only uploads, submit a DOCX version instead.

• The internship instructions state that a GitHub project URL is compulsory for technical/software-related tasks. Create and publish the project repository, then enter its actual URL. Do not use a placeholder or an invented URL.

• Replace the candidate sensor fields and assumptions with the actual dataset columns, prediction horizon, city/location scope, and sensor details once confirmed.

• Do not claim that model training, measured performance, or live deployment has been completed unless those activities have actually been performed. This document is a strategy and design report.

Suggested References / Tools

• Python documentation: https://docs.python.org/3/

• pandas documentation: https://pandas.pydata.org/docs/

• scikit-learn user guide: https://scikit-learn.org/stable/user_guide.html

• Matplotlib documentation: https://matplotlib.org/stable/users/index.html

- Streamlit documentation: https://docs.streamlit.io/